Date : 20 avril 2026

Projet : central-esp32

Objectif : centrale ESP32 pour gérer les coins, les postes de jeu, le Wi-Fi, la sécurité admin et l'interface web locale.

Accès web prévu : `http://gameroom.local` ou adresse IP locale de l'ESP32.

1. Résumé exécutif

Le projet est une application Arduino/ESP32 organisée en services C++ autour d'un serveur web embarqué. Le rôle de la carte centrale est de recevoir les impulsions d'un monnayeur, convertir ces impulsions en crédits, assigner des crédits aux postes esclaves, suivre leur statut, offrir une interface d'administration et permettre une configuration Wi-Fi autonome.

Le code est globalement bien séparé : chaque domaine principal dispose d'un fichier `.h` et d'un fichier `.cpp`. Les fichiers les plus volumineux sont `WebPage.cpp`, qui contient l'interface HTML/CSS/JS, `WebRoutes.cpp`, qui expose les routes HTTP/API, et `WifiProvisioning.cpp`, qui gère le portail Wi-Fi.

> Pendant ce scan, le code a été légèrement organisé : valeurs par défaut centralisées dans `AppConfig.h`, suppression d'une allocation dynamique inutile dans `WebRoutes.cpp`, et conservation de la correction anti-blocage sur la découverte réseau.

2. Fonctionnalités principales

Fonctionnalité	Description	Modules concernés
Interface web admin	Dashboard, login, gestion des postes, crédits, logs, configuration, export/import.	`WebPage.cpp`, `WebRoutes.cpp`, `AuthService.cpp`
Accès local nommé	Annonce mDNS sous `gameroom.local` avec service HTTP sur le port 80.	`WifiService.cpp`, `AppConfig.h`
Configuration Wi-Fi	Si aucun Wi-Fi n'est configuré, création d'un point d'accès `GameRoom-Setup` et portail captif.	`WifiProvisioning.cpp`, `WifiService.cpp`
Gestion du monnayeur	Lecture des impulsions sur le GPIO 27, anti-rebond, conversion en crédits.	`CoinService.cpp`, `AppState.h`
Gestion des postes	Ajout, modification, suppression, assignation de crédits, arrêt manuel, rafraîchissement du statut.	`PostService.cpp`, `SlaveClient.cpp`
Découverte réseau	Scan progressif du réseau local pour trouver des postes esclaves qui répondent à `/status`.	`DiscoveryService.cpp`

Persistance	Sauvegarde des postes, crédits, configuration, mot de passe admin et token API dans `Preferences`.	`StorageService.cpp`
Logs système	Journal en mémoire limité aux derniers événements pour diagnostic depuis le dashboard.	`LogService.cpp`

3. Architecture du code

```
central-esp32.ino
  -> initialise stockage, authentification, Wi-Fi, services et serveur web
  -> boucle principale : serveur HTTP, monnayeur, statuts postes, découverte réseau

AppConfig.h
  -> constantes globales : Wi-Fi, mDNS, port, GPIO, timeouts, valeurs par défaut

AppState.h / Post.h
  -> modèles de données en mémoire

Services métier
  -> AuthService : sessions et token API
  -> CoinService : impulsions monnayeur et crédits
  -> PostService : règles métier des postes
  -> SlaveClient : client HTTP vers les postes esclaves
  -> DiscoveryService : scan réseau progressif
  -> StorageService : sauvegarde Preferences et JSON
  -> LogService : événements internes
  -> WifiService / WifiProvisioning : connexion Wi-Fi, mDNS, portail captif

Web
  -> WebRoutes : endpoints HTTP/API
  -> WebPage : pages HTML embarquées
```

4. Description des fichiers

Fichier	Rôle	Commentaires
`central-esp32.ino`	Point d'entrée Arduino.	Orchestre les services. Reste volontairement court.
`AppConfig.h`	Configuration centrale.	Bon endroit pour changer `gameroom`, le port, GPIO et timeouts.
`AppState.h`	État global en RAM.	Contient postes, crédits, logs, auth, compteurs interrupt.
`Post.h`	Modèle d'un poste.	ID, nom, IP, statut, relais, temps restant.
`AuthService.*`	Authentification.	Session cookie `ESPSESSION` et bearer token API.
`CoinService.*`	Monnayeur.	Utilise interruption GPIO et sauvegarde les crédits.

`PostService.*`	Gestion métier postes.	Validation, assignation de coins, stop, refresh progressif.
`SlaveClient.*`	Client HTTP poste esclave.	Appelle `/command` et `/status`.
`DiscoveryService.*`	Découverte automatique.	Scan une IP par cycle pour préserver la réactivité web.
`StorageService.*`	Persistance.	Preferences + sérialisation JSON des postes/config.
`LogService.*`	Journal système.	Garde les 100 derniers logs en mémoire.
`WifiService.*`	Wi-Fi normal et mDNS.	Démarre `gameroom.local`.
`WifiProvisioning.*`	Mode configuration Wi-Fi.	AP, DNS captif, scan réseaux, sauvegarde credentials.
`WebRoutes.*`	Routes HTTP.	API JSON et pages web.
`WebPage.*`	HTML/CSS/JS embarqué.	Dashboard admin et page login.

5. Flux de démarrage

1. `setup()` initialise le port série, le stockage et charge `AppState`.
2. `AuthService::begin()` initialise la couche auth.
3. `WifiService::begin()` tente une connexion avec les identifiants Wi-Fi sauvegardés.
4. Si la connexion échoue ou si aucun identifiant n'existe, le mode AP/provisioning démarre.
5. Si le Wi-Fi fonctionne, mDNS démarre : `http://gameroom.local`.
6. Les services monnayeur, postes et routes web démarrent.
7. `loop()` sert les requêtes web et exécute les tâches périodiques.

6. API HTTP exposée

Route	Méthode	But	Auth
`/`	GET	Dashboard admin.	Cookie session
`/login`	GET/POST	Page de login et connexion.	POST public
`/logout`	POST	Déconnexion.	Session
`/posts`	GET	État des postes, crédits, configuration rapide.	Session ou token API
`/add-post`	POST	Ajouter un poste.	Session ou token API
`/post/update`	POST	Modifier nom/IP d'un poste.	Session ou token API
`/post/delete`	POST	Supprimer un poste.	Session ou token API
`/post/ping`	POST	Tester un poste.	Session ou token API
`/assign`	POST	Affecter des coins à un poste.	Session ou token API
`/stop`	POST	Arrêter une session poste.	Session ou token API

`/config`	POST	Modifier durée/impulsions/crédits.	Session ou token API
`/config/export`	GET	Exporter la configuration JSON.	Session ou token API
`/config/import`	POST	Importer une configuration JSON.	Session ou token API
`/auth/password`	POST	Changer le mot de passe admin.	Session ou token API
`/auth/token/regenerate`	POST	Régénérer le token API.	Session ou token API
`/logs`	GET	Lire les logs.	Session ou token API
`/logs/clear`	POST	Vider les logs.	Session ou token API
`/discovery/run`	POST	Lancer la découverte réseau progressive.	Session ou token API
`/wifi/reset`	POST	Supprimer les credentials Wi-Fi et redémarrer.	Session ou token API

7. Données et persistance

Le projet utilise `Preferences` ESP32 avec plusieurs namespaces :

Namespace	Clés	Contenu
`appcfg`	`coinDur`, `pulseCoin`, `availCoins`	Configuration et crédits.
`auth`	`adminPwd`, `apiToken`	Sécurité admin/API.
`posts`	`items`	Liste JSON des postes.
`wifi`	`ssid`, `password`	Identifiants Wi-Fi.

8. Documentation des services

AuthService

Fournit `login`, `logout`, vérification cookie et bearer token. Le token API est généré avec `esp_random()`. La session web est stockée en mémoire : elle disparaît au redémarrage, ce qui est acceptable pour une console admin embarquée.

CoinService

Configure le GPIO `AppConfig::COIN_PIN` en `INPUT_PULLUP` et attache une interruption sur front descendant. L'interruption incrémente `pulseCount` après anti-rebond. `update()` convertit les impulsions en crédits selon `pulsesPerCoin`.

PostService

Valide l'unicité des ID/IP, envoie les commandes aux postes esclaves, sauvegarde les changements et rafraîchit les statuts. Le rafraîchissement est progressif pour éviter qu'un poste hors ligne bloque toute l'interface.

SlaveClient

Utilise `HTTPClient` pour communiquer avec les postes : `POST http://IP/command` et `GET http://IP/status`. Le timeout standard est `AppConfig::SLAVE_STATUS_TIMEOUT_MS`, et la découverte utilise un timeout plus court.

DiscoveryService

Le scan réseau est lancé explicitement par `requestScan()`. Ensuite `run()` teste une adresse IP par passage dans la boucle principale. Cette organisation empêche le serveur web de rester bloqué pendant un scan complet du subnet.

WifiService et WifiProvisioning

`WifiService` gère la connexion normale et mDNS. `WifiProvisioning` gère le mode AP avec portail captif : page de configuration Wi-Fi, scan des réseaux et sauvegarde des credentials.

9. Organisation appliquée pendant le scan

- Ajout de constantes par défaut dans `AppConfig.h` : mot de passe admin, durée par coin, impulsions par coin, crédits initiaux.
- `AppState.h` utilise maintenant ces constantes au lieu de valeurs codées en dur.
- `StorageService.cpp` utilise les mêmes constantes lors du chargement Preferences.
- `SlaveClient.h` utilise `AppConfig::SLAVE_STATUS_TIMEOUT_MS` comme timeout par défaut.
- `WebRoutes.cpp` remplace `new const char*[2]` par un tableau statique `AUTH_HEADERS`.
- La correction précédente de découverte réseau progressive a été conservée et documentée.

10. Points d'attention

> **Compilation non vérifiée localement :** `arduino-cli` n'est pas installé dans le shell courant. Il faut compiler depuis l'IDE Arduino ou installer `arduino-cli` pour une vérification automatisée.

- **Sécurité :** le mot de passe par défaut est `admin1234`. Il doit être changé dès la première installation.
- **mDNS :** `gameroom.local` dépend du support mDNS du client. Sur certains Windows, Bonjour/mDNS peut être nécessaire.
- **Mémoire :** les documents JSON de 8192 octets dans `WebRoutes.cpp` sont corrects pour un petit parc de postes, mais doivent être surveillés si la liste grandit.
- **Interface embarquée :** `WebPage.cpp` contient beaucoup de HTML/JS dans un raw string. C'est simple pour Arduino, mais plus difficile à maintenir qu'un fichier généré.

11. Recommandations

1. Compiler et uploader après les changements, puis tester `http://gameroom.local`.
2. Depuis le dashboard, vérifier que `/posts` et `/logs` répondent sans blocage.
3. Lancer une découverte réseau et confirmer que la page reste responsive pendant le scan.
4. Changer le mot de passe admin et sauvegarder le token API généré.
5. Si le projet grandit, extraire progressivement le JavaScript/CSS de `WebPage.cpp` vers un outil de génération ou des fichiers servis depuis LittleFS/SPIFFS.

12. Checklist de test manuel

Test	Résultat attendu
Démarrage sans Wi-Fi sauvegardé	AP `GameRoom-Setup` visible, portail accessible.
Démarrage avec Wi-Fi sauvegardé	Connexion au réseau local, logs avec IP et URL `gameroom.local`.
Login admin	Cookie `ESPSESSION` créé, redirection vers dashboard.
Chargement dashboard	Stats, postes et logs visibles, sans attente infinie.
Ajouter poste	Poste sauvegardé et visible après rechargement.
Assigner coin	Crédits diminuent et commande envoyée au poste.
Poste hors ligne	Statut passe à `offline` sans bloquer la page.
Découverte réseau	Scan progressif, logs de début/fin, interface responsive.
Export/import config	JSON exporté puis réimporté sans erreur.
Reset Wi-Fi	Credentials supprimés, redémarrage en mode configuration.